

Computation Graphs, Backpropagation and Autodiff

How gradients flow through neural networks

Prof. Nipun Batra

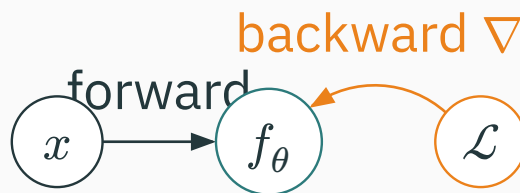
ES 667 · Deep Learning · IIT Gandhinagar

Backprop vocabulary

The goal: two passes over one graph

v

Training is a **forward** pass and a **backward** pass over the same computation graph:



forward: $x \rightarrow \mathcal{L}$ • **backward:** $\mathcal{L} \rightarrow \nabla_\theta \mathcal{L}$

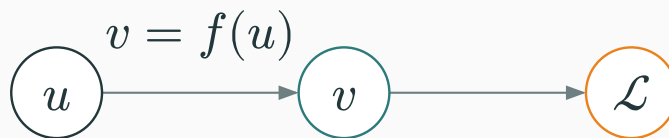
Every update step is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).$$

A modern network has $P = 10^7$ to 10^{11} parameters.

We need **all** P partial derivatives $\partial \mathcal{L} / \partial \theta_j$ — cheaply, and exactly. Backprop delivers them in **one backward pass**.

Zoom in on a single edge of the graph: $u \rightarrow v \rightarrow \mathcal{L}$.



We want $\bar{u} := \partial \mathcal{L} / \partial u$, and we are **given** $\bar{v} := \partial \mathcal{L} / \partial v$ from the node ahead.

The chain rule splits the work into two pieces we already have:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial u}}_{\text{downstream}} = \underbrace{\frac{\partial \mathcal{L}}{\partial v}}_{\text{upstream}} \cdot \underbrace{\frac{\partial v}{\partial u}}_{\text{local}}$$

$$\text{downstream} = \text{upstream} \times \text{local}$$

Upstream comes from the node ahead. **Local** is the derivative of **this** node alone. Their product flows **downstream**.

Write $\bar{u} = \partial \mathcal{L} / \partial u$. Then every node obeys

$$\bar{u} = \bar{v} \cdot \frac{\partial v}{\partial u}.$$

The node only needs to know its **own** local derivative $\partial v / \partial u$. It multiplies whatever gradient $\bar{v}$ arrives from above — no global bookkeeping.

Node $v = u^2$, so the local derivative is $\partial v / \partial u = 2u$. Suppose $u = 3$ (forward) and the upstream gradient is $\bar{v} = 7$. Then

$$\bar{u} = \bar{v} \cdot 2u = 7 \cdot 6 = 42.$$

downstream $\bar{u} = 42$

Addition node: copy the gradient

D

Node $v = a + b$. Local derivatives $\partial v / \partial a = 1$, $\partial v / \partial b = 1$.

$$\bar{a} = \bar{v} \cdot 1 = \bar{v}, \quad \bar{b} = \bar{v} \cdot 1 = \bar{v}.$$

+ **copies** the upstream gradient to every input

Multiplication node: swap the inputs

D

Node $v = ab$. Local derivatives $\partial v / \partial a = b$, $\partial v / \partial b = a$.

$$\bar{a} = \bar{v} \cdot b, \quad \bar{b} = \bar{v} \cdot a.$$

× **sends the other input**

If $a = 2, b = 5, \bar{v} = 3$: then $\bar{a} = 3 \cdot 5 = 15$ and $\bar{b} = 3 \cdot 2 = 6$.

Node $v = \varphi(u)$ (elementwise nonlinearity). Local derivative $\partial v / \partial u = \varphi'(u)$.

$$\bar{u} = \bar{v} \cdot \varphi'(u).$$

sigmoid σ : $\varphi'(u) = \sigma(u)(1 - \sigma(u))$

ReLU: $\varphi'(u) = \mathbb{1}[u > 0]$

The rule table — memorize this

v

Forward op	Local grad	Backward update
$v = a + b$	$1, 1$	$\bar{a} += \bar{v}, \bar{b} += \bar{v}$
$v = ab$	b, a	$\bar{a} += \bar{v}b, \bar{b} += \bar{v}a$
$v = u^2$	$2u$	$\bar{u} += \bar{v} \cdot 2u$
$v = \varphi(u)$	$\varphi'(u)$	$\bar{u} += \bar{v}\varphi'(u)$

Use +=, not = — a variable feeding several nodes accumulates gradient from **all** of them.

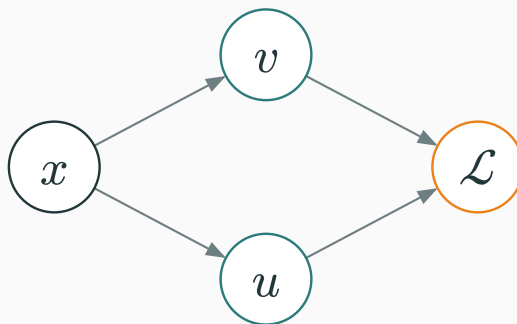
Gradients accumulate at branches

If x influences $\mathcal{L}$ through several intermediate variables $u_1, \dots, u_k$:

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{i=1}^k \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial x}.$$

gradients from every path add

When x fans out, its gradient is the **sum** over out-edges:



$$\bar{x} = \bar{x}_{\text{via } u} + \bar{x}_{\text{via } v}.$$

Let $u = x^2$, $v = 3x$, $\mathcal{L} = u + v = x^2 + 3x$. By direct calculus:

$$\frac{d\mathcal{L}}{dx} = 2x + 3, \quad \text{at } x = 4: \quad 2(4) + 3 = 11.$$

Now let us get the **same** answer by backprop.

Forward at $x = 4$: $u = 16$, $v = 12$, $\mathcal{L} = 28$. **Backward**: seed $\overline{\mathcal{L}} = 1$; add node gives $\overline{u} = 1$, $\overline{v} = 1$.

$$\overline{x}_{\text{via } u} = \overline{u} \cdot 2x = 1 \cdot 8 = 8, \quad \overline{x}_{\text{via } v} = \overline{v} \cdot 3 = 3.$$

$$\overline{x} = 8 + 3 = 11. \quad \checkmark$$

Both paths write into the **same** $\bar{x}$. If the second overwrote the first, we would lose the 8.

In PyTorch, `.grad` **accumulates** across `backward()` calls — that is exactly this rule. You must `optimizer.zero_grad()` each step, or gradients from old steps pile up.

The central worked example

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$. Concrete numbers:

$$x = 3, \quad w = 2, \quad b = 1, \quad y = 10.$$

Four scalars, one loss. We will get $\partial \mathcal{L}$ w.r.t. **all four** by hand, then check with PyTorch.

Break into primitives

Decompose the loss into elementary nodes:

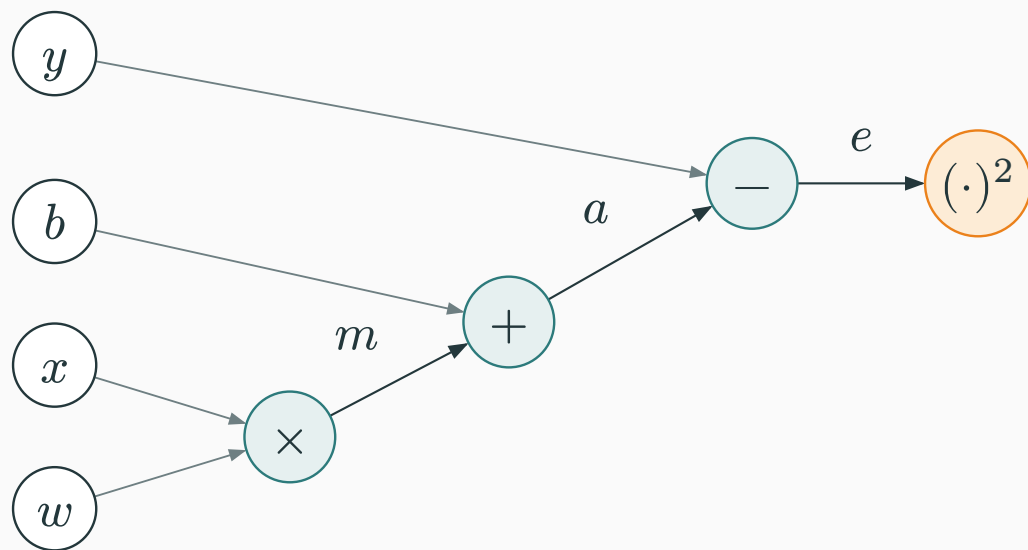
$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

$\times$, then $+$, then $-$, then $(\cdot)^2$

Each is one of the nodes from our rule table.

The computation graph

v



inputs (ink) $\rightarrow$ ops $\times, +, -$ (teal) $\rightarrow$ loss $(\cdot)^2$ (orange)

Node	Rule	Value
$m = wx$	$2 \cdot 3$	6
$a = m + b$	$6 + 1$	7
$e = a - y$	$7 - 10$	-3
$\mathcal{L} = e^2$	$(-3)^2$	9

$\mathcal{L} = 9$ — now run the graph **backward**

Seed the output: $\overline{\mathcal{L}} = \partial \mathcal{L} / \partial \mathcal{L} = 1$. Square node $\mathcal{L} = e^2$, local $\partial \mathcal{L} / \partial e = 2e$:

$$\overline{e} = \overline{\mathcal{L}} \cdot 2e = 1 \cdot 2(-3) = -6.$$

Backward: the subtraction node

D

Node $e = a - y$, locals $\partial e / \partial a = 1$, $\partial e / \partial y = -1$:

$$\bar{a} = \bar{e} \cdot 1 = -6, \quad \bar{y} = \bar{e} \cdot (-1) = 6.$$

The target y already has a gradient — we would use $\bar{y}$ if y were itself a learned quantity.

Node $a = m + b$ **copies** its gradient:

$$\overline{m} = \overline{a} = -6, \quad \overline{b} = \overline{a} = -6.$$

So $\overline{b} = -6$ is one of our final answers.

Node $m = wx$ **sends the other input:**

$$\bar{w} = \bar{m} \cdot x = -6 \cdot 3 = -18, \quad \bar{x} = \bar{m} \cdot w = -6 \cdot 2 = -12.$$

$$\bar{w} = -18, \quad \bar{x} = -12$$

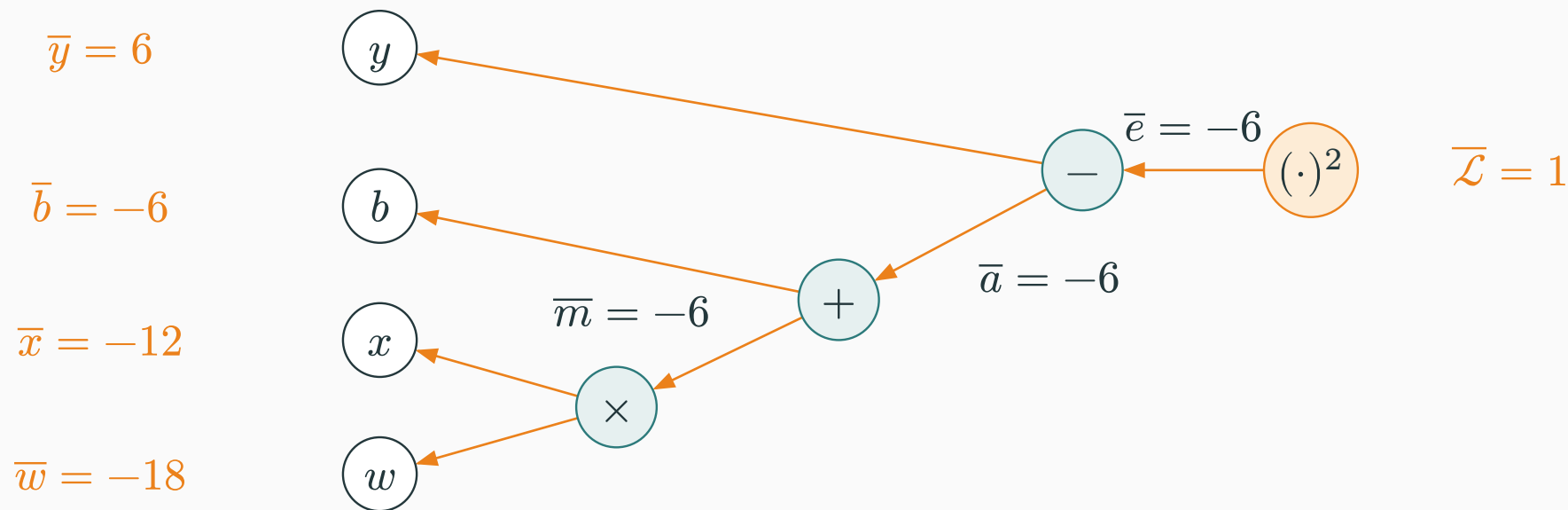
$\partial \mathcal{L} / \partial w$	$\partial \mathcal{L} / \partial b$	$\partial \mathcal{L} / \partial x$	$\partial \mathcal{L} / \partial y$
-18	-6	-12	6

One backward sweep produced **all four** partial derivatives — no formula re-derivation per parameter.

The backward pass, visualized

v

Seed $\bar{\mathcal{L}} = 1$ at the loss; each **orange arrow** carries **upstream $\times$ local** one node to the left:



same graph as the forward pass — every arrow reversed, adjoints filling in right to left

Differentiate $\mathcal{L} = (wx + b - y)^2$ directly. With $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

backprop = calculus — but mechanical & reusable

One gradient-descent step

D

Take $\eta = 0.1$ and update:

$$w \leftarrow 2 - 0.1(-18) = 3.8, \quad b \leftarrow 1 - 0.1(-6) = 1.6.$$

New prediction: $\hat{y} = 3.8 \cdot 3 + 1.6 = 13$.

Target is $y = 10$; we jumped from 6 to 13 — we **overshot**. Lecture 4 is about choosing η and the update rule so this is stable.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Build a scalar computation graph, run the forward pass, then click **backward** to watch each adjoint $\bar{u}$ fill in — exactly the $(wx + b - y)^2$ example above.

Q. Which node's local derivative changes if we swap squared-error for absolute error $|e|$?

From scalar graph to a neuron

A single neuron

Vector input, one output:

$$z = w^\top x + b, \quad a = \varphi(z), \quad \mathcal{L} = \ell(a, y).$$

same three nodes: affine → activation → loss

Let $\delta := \bar{z} = \partial \mathcal{L} / \partial z$. Through $a = \varphi(z)$:

$$\delta = \bar{a} \cdot \varphi'(z), \quad \text{where } \bar{a} = \frac{\partial \mathcal{L}}{\partial a}.$$

δ is the **error signal at the pre-activation**. Everything below the neuron is expressed through this one scalar.

Through $z = w^\top x + b$ with $\bar{z} = \delta$:

$$\nabla_w \mathcal{L} = \delta x, \quad \frac{\partial \mathcal{L}}{\partial b} = \delta, \quad \bar{x} = \delta w.$$

Parameter gradient = error δ times the **input** x . This “ δ times input” shape reappears at every layer.

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$.

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1 - y}{1 - a}, \quad \frac{\partial a}{\partial z} = a(1 - a).$$

Multiply — the denominators cancel:

$$\bar{z} = \frac{\partial \mathcal{L}}{\partial a} \cdot a(1 - a) = a - y.$$

$$\bar{z} = a - y \text{ — prediction minus target}$$

For multi-class, $p = \text{softmax}(z)$ and $\mathcal{L} = -\log p_y$:

$$\nabla_z \mathcal{L} = p - y,$$

where y is the one-hot target.

$\bar{z} = p - y$ — the identical “prediction – target” form

Choosing the loss to **match** the output nonlinearity makes the output-layer gradient trivially simple.

The dense layer

A dense layer maps $x \in \mathbb{R}^n$ to $z \in \mathbb{R}^m$:

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m.$$

Read the shapes off the equation: W has one row per output, one column per input.

Given upstream $\bar{z} \in \mathbb{R}^m$:

$$\nabla_W \mathcal{L} = \bar{z} x^\top \quad (m \times n), \quad \nabla_b \mathcal{L} = \bar{z}, \quad \bar{x} = W^\top \bar{z} \quad (n).$$

$$\nabla_W = \bar{z} x^\top \cdot \bar{x} = W^\top \bar{z}$$

Stack N examples as rows: $X \in \mathbb{R}^{N \times n}$, $Z = XW^\top + \mathbf{1}b^\top$. Given $\bar{Z} \in \mathbb{R}^{N \times m}$:

$$\nabla_W \mathcal{L} = \bar{Z}^\top X, \quad \nabla_b \mathcal{L} = \sum_{i=1}^N \bar{Z}_i.$$

Summing rows of $\bar{Z}$ for ∇_b is just the branch rule: b is **shared** across all N examples, so its gradients accumulate.

Quantity	Shape	Must match
W	$m \times n$	∇_W
b	m	∇_b
x	n	$\bar{x}$
z	m	$\bar{z}$

gradient shape = parameter shape — always

Backprop through a two-layer MLP

$$z^{(1)} = W_1 x + b_1, \quad h = \varphi(z^{(1)}),$$

$$z^{(2)} = W_2 h + b_2, \quad p = \text{softmax}(z^{(2)}), \quad \mathcal{L} = -\log p_y.$$

affine → activation → affine → softmax → CE

The forward graph

v



backprop reverses every arrow, right to left

Softmax + CE gives the clean output-layer signal:

$$\bar{z}^{(2)} = p - y.$$

Then, using the dense-layer rules:

$$\nabla_{W_2} \mathcal{L} = \bar{z}^{(2)} h^\top, \quad \bar{h} = W_2^\top \bar{z}^{(2)}.$$

Through the activation $h = \varphi(z^{(1)})$ (elementwise):

$$\bar{z}^{(1)} = \bar{h} \odot \varphi'(z^{(1)}).$$

Then the first affine layer:

$$\nabla_{W_1} \mathcal{L} = \bar{z}^{(1)} x^\top, \quad \nabla_{b_1} \mathcal{L} = \bar{z}^{(1)}.$$

$\odot$ is elementwise (Hadamard) — the activation Jacobian is diagonal.

Forward	Backward
$z^{(1)} = W_1 x + b_1$	$\bar{z}^{(1)} = \bar{h} \odot \varphi'(z^{(1)})$
$h = \varphi(z^{(1)})$	$\nabla_{W_1} = \bar{z}^{(1)} x^\top, \nabla_{b_1} = \bar{z}^{(1)}$
$z^{(2)} = W_2 h + b_2$	$\bar{h} = W_2^\top \bar{z}^{(2)}$
$p = \text{softmax}(z^{(2)})$	$\nabla_{W_2} = \bar{z}^{(2)} h^\top, \nabla_{b_2} = \bar{z}^{(2)}$
$\mathcal{L} = -\log p_y$	$\bar{z}^{(2)} = p - y$

same two moves at every layer: $\delta x^\top$ for weights, $W^\top \delta$ downstream

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Step through a small MLP's forward pass, then watch $\bar{z}^{(2)} = p - y$ propagate back through W_2 , the activation, and W_1 — the adjoints light up layer by layer.

Q. Why does the **same** $\bar{z}$ appear in both ∇_W and $\bar{x}$ for a layer?

Automatic differentiation

Backprop is reverse-mode autodiff

Backprop is not two things you might guess:

not symbolic differentiation — no giant closed-form expression

not finite differences — no ε -perturbations

it is reverse-mode automatic differentiation: the chain rule applied to the executed program

Why reverse mode?

D

A loss is a map $f : \mathbb{R}^P \rightarrow \mathbb{R}$: **many** inputs, **one** output.

Forward mode: one pass **per input** $\rightarrow O(P)$.

Reverse mode: one pass **per output** $\rightarrow O(1)$.

$P \approx 10^9$ **inputs, one scalar loss $\Rightarrow$ reverse mode wins**

What PyTorch / JAX actually do

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);
- **stores** intermediate values (needed by local derivatives);
- on `backward()`: seeds $\overline{\mathcal{L}} = 1$, applies each op's local rule, and **accumulates** into `.grad`.

you write only the forward pass – the backward pass is generated

PyTorch: the scalar example

I

```
import torch
x = torch.tensor(3.0)
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(10.0)

L = (w*x + b - y)**2      # forward: L = 9
L.backward()              # reverse-mode autodiff

print(L.item())           # 9.0
print(w.grad, b.grad)     # tensor(-18.) tensor(-6.)
```

matches our hand computation exactly

Gradients accumulate in frameworks

.grad is **added to**, never reset, by backward(). The standard loop is:

```
for x, y in loader:
    optimizer.zero_grad()      # clear old gradients
    loss = criterion(model(x), y)
    loss.backward()            # accumulate this batch's gradients
    optimizer.step()           # theta <- theta - eta * grad
```

Forget `zero_grad()` and you sum gradients across steps — the same += branch rule, now biting you.

Gradient checking & gradient flow

To **debug** an analytic gradient, compare against a central difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\theta + \varepsilon e_j) - \mathcal{L}(\theta - \varepsilon e_j)}{2\varepsilon}.$$

Great for **unit tests** ($\varepsilon \approx 10^{-5}$), far too slow for training — one loss evaluation **per parameter**.

Stack L layers $h_0 \rightarrow h_1 \rightarrow \dots \rightarrow h_L$. The chain rule multiplies Jacobians:

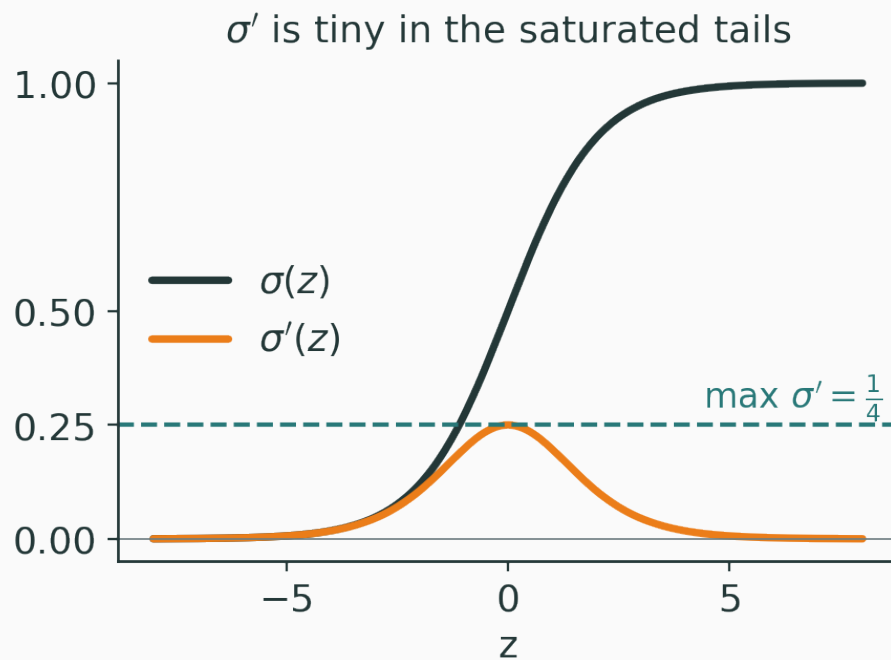
$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^L \frac{\partial h_\ell}{\partial h_{\ell-1}}.$$

a product of L terms — it can vanish or explode

Sigmoid saturation

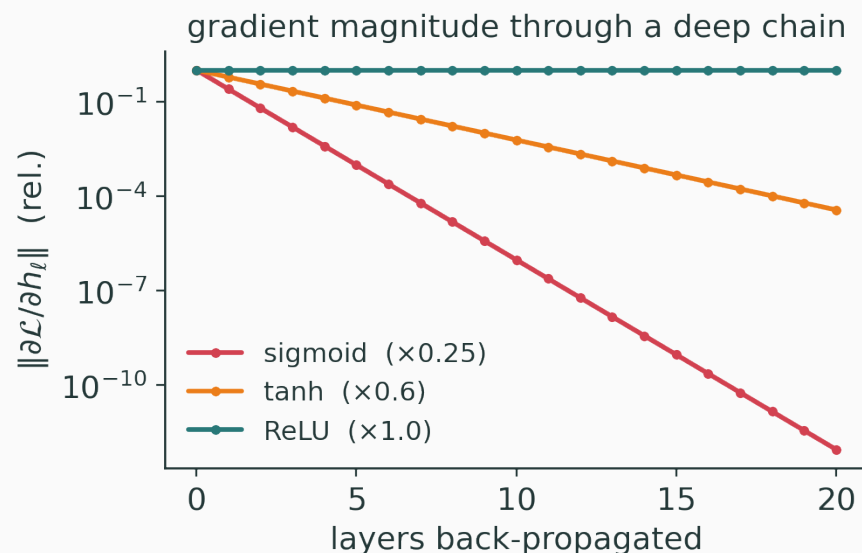
v

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:



each sigmoid layer multiplies the gradient by at most $1/4$

Multiply many small local derivatives $\rightarrow$ the signal **decays** with depth:



Sigmoid decays fast; ReLU (local slope 0 or 1) keeps gradients alive.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Q. If gradients shrink by $\times 1/4$ per layer, how deep before they underflow to zero?

Summary

Seed $\bar{\mathcal{L}} = 1$, then walk the graph **backward**, at each node computing
downstream = upstream $\times$ local,
and **adding** gradients wherever a variable branched.

The formulas to keep

v

Setting	Backward rule
scalar node	$\bar{u} = \bar{v} \cdot \partial v / \partial u$
dense layer	$\nabla_W = \bar{z} x^\top, \quad \bar{x} = W^\top \bar{z}$
activation	$\bar{z} = \bar{a} \odot \varphi'(z)$
softmax + CE	$\bar{z} = p - y$
branch point	gradients add

What comes next

We can now compute $\nabla_{\theta} \mathcal{L}$ for **any** network, exactly and cheaply.

Next (Lecture 4): how to **use it — SGD, momentum, Adam**

Backprop gives the **direction**; optimization decides the **step**.

We can compute $\nabla_{\theta} \mathcal{L}$.

Next: how to **use** it — SGD, momentum, Adam.